Terraform on Google Cloud with Variables: Lab Guide

Level: Introductory. Estimated time: 75 minutes. Prerequisite: the basic Terraform VM lab (provider, plan, apply, destroy).

Overview

In the first lab every value (project, zone, machine type, tags) was typed directly into main.tf. That works for one VM in one project, but the moment you want a second environment or a different machine size you have to edit the code itself. This lab rebuilds the same Compute Engine VM, then moves every hard-coded value into input variables, supplies them from a tfvars file, overrides them from the command line, adds validation, and publishes useful values as outputs. The lab ends by destroying everything, as before.

Objectives

- Separate configuration into main.tf, variables.tf, terraform.tfvars and outputs.tf.
- Declare input variables with types, descriptions and default values.
- Supply variable values in four ways and understand which one wins.
- Use a list variable for network tags and a map variable for labels.
- Reject bad input with a validation block.
- Read results with output values and terraform output.
- Change and destroy the VM using only variable changes, not code edits.

Task 1. Prepare Cloud Shell

Sign in to the console with the lab account, open Cloud Shell, and confirm you are in the lab project:

```
gcloud auth list
gcloud config list project
```

If Terraform is not installed yet, install it the same way as in the previous lab (the ~/.customize_environment script), then check it:

```
terraform --version
```

Create a fresh working folder so this lab does not share state with the previous one:

```
mkdir tf-variables-lab && cd tf-variables-lab
touch main.tf variables.tf terraform.tfvars outputs.tf
```

Note your project ID, region and zone from the lab panel. You will need them in Task 3.

Task 2. Declare the input variables

Open variables.tf in the Cloud Shell Editor. Each variable block names an input, says what type of value it accepts and, optionally, gives it a default. A variable without a default is required:

Terraform will refuse to plan until it gets a value.

```
variable "project_id" {
  description = "Google Cloud project that will hold the VM"
  type        = string
}

variable "region" {
  description = "Default region for regional resources"
  type        = string
  default     = "us-central1"
}

variable "zone" {
  description = "Zone where the VM is created"
  type        = string
  default     = "us-central1-a"
}

variable "instance_name" {
  description = "Name of the Compute Engine VM"
  type        = string
  default     = "terraform-vm"
}

variable "machine_type" {
  description = "Machine type for the VM"
  type        = string
  default     = "e2-micro"
}

variable "image" {
  description = "Boot disk image in project/family form"
  type        = string
  default     = "debian-cloud/debian-12"
}

variable "network_tags" {
  description = "Network tags applied to the VM"
  type        = list(string)
  default     = []
}

variable "labels" {
  description = "Key/value labels applied to the VM"
  type        = map(string)
  default     = {}
}
```

Save the file. Notice that project_id is the only required variable, because there is no sensible

default for it.

Task 3. Write the configuration using the variables

Open main.tf. Every place that used to contain a literal value now refers to a variable with the var. prefix.

```
terraform {
  required_providers {
    google = {
      source = "hashicorp/google"
    }
  }
}

provider "google" {
  project = var.project_id
  region  = var.region
  zone    = var.zone
}

resource "google_compute_instance" "vm" {
  name         = var.instance_name
  machine_type = var.machine_type
  zone         = var.zone
  tags         = var.network_tags
  labels       = var.labels

  boot_disk {
    initialize_params {
      image = var.image
    }
  }

  network_interface {
    network = "default"
    access_config {}
  }

  allow_stopping_for_update = true
}
```

allow_stopping_for_update is included from the start this time, since the previous lab showed that a machine type change fails without it.

Initialize the folder:

```
terraform init
```

See what happens with a missing required variable

Run a plan before giving project_id a value:

```
terraform plan
```

Terraform stops and prompts you to type a value for var.project_id. Press Ctrl+C to cancel. Typing values at a prompt is easy to get wrong and impossible to repeat exactly, so the next task stores them in a file instead.

Task 4. Supply values with terraform.tfvars

Terraform automatically loads a file named terraform.tfvars from the working folder. Open it and add your values (replace the placeholders with the ones from the lab panel):

```
project_id    = "YOUR_PROJECT_ID"
region        = "YOUR_REGION"
zone          = "YOUR_ZONE"
instance_name = "terraform-vm"
machine_type  = "e2-micro"
```

Save the file and plan again:

```
terraform plan
```

There is no prompt this time. The plan shows one VM to create, and the name, zone and machine type in the plan match the values in terraform.tfvars. Build it:

```
terraform apply
```

Type yes when asked. In the console, go to Compute Engine, VM instances and confirm that terraform-vm exists in the zone you chose.

Task 5. Expose results with outputs

Outputs print selected values after an apply and make them easy to read later or pass to other tools. Open outputs.tf:

```
output "instance_name" {
  description = "Name of the VM"
  value       = google_compute_instance.vm.name
}

output "machine_type" {
  description = "Current machine type"
  value       = google_compute_instance.vm.machine_type
}

output "internal_ip" {
  description = "Private IP address of the VM"
  value       =
google_compute_instance.vm.network_interface[0].network_ip
}

output "external_ip" {
  description = "Ephemeral public IP address of the VM"
  value       =
google_compute_instance.vm.network_interface[0].access_config[0].nat_i
```

```
p
}
```
Apply again. Nothing in the infrastructure changes, but Terraform records the outputs:
```
terraform apply
terraform output
terraform output -raw external_ip
```
The -raw form prints only the value, which is handy in scripts, for example to ping or SSH to the VM.

Task 6. Change the VM through variables only

In this task you never edit main.tf. Every change goes through variable values.

Add tags and labels with list and map variables

Append these lines to terraform.tfvars:
```
network_tags = ["web", "dev"]

labels = {
  env   = "dev"
  owner = "student"
}
```
Run a plan. The resource is marked with ~ (update in place), and the plan lists the two new tags and two new labels. Apply it and confirm with yes. The VM keeps running throughout.

Resize the VM

Change one line in terraform.tfvars:
```
machine_type = "e2-medium"
```
Plan and apply. Because allow_stopping_for_update is already true, Terraform stops the VM, changes the machine type and starts it again. Check the result:
```
terraform output machine_type
```

Task 7. Override values from the command line and environment

Values can come from several places. When the same variable is set in more than one, the later source in this list wins:

1. The default in variables.tf.
2. An environment variable named TF_VAR_ followed by the variable name.
3. terraform.tfvars, then any *.auto.tfvars files.
4. A -var-file option on the command.
5. A -var option on the command.

Try a one-off override

Preview a larger machine without touching any file:

```
terraform plan -var="machine_type=e2-standard-2"
```

The plan shows the change from e2-medium to e2-standard-2 because -var outranks terraform.tfvars. Do not apply it; this was only a preview.

Try an environment variable

```
export TF_VAR_instance_name="env-var-vm"
terraform plan
```

The plan does not rename the VM. terraform.tfvars also sets instance_name, and the file outranks the environment variable. Remove the line instance_name = "terraform-vm" from terraform.tfvars and plan again: now the environment value is used, and because a VM's name cannot change in place, the plan shows -/+ (destroy and recreate). Do not apply. Put the line back in terraform.tfvars and clear the environment variable:

```
unset TF_VAR_instance_name
```

Use a separate file per environment

Create prod.tfvars with different values:

```
machine_type = "e2-standard-2"
network_tags = ["web", "prod"]
labels = {
  env   = "prod"
  owner = "student"
}
```

Preview it:

```
terraform plan -var-file="prod.tfvars"
```

This is how one set of .tf files can serve dev, test and prod: the code stays the same and only the tfvars file changes. Do not apply.

Task 8. Validate input

Variables can reject values before Terraform contacts Google Cloud. In variables.tf, replace the machine_type block with this version:

```
variable "machine_type" {
  description = "Machine type for the VM (E2 family only)"
  type        = string
  default     = "e2-micro"

  validation {
    condition     = startswith(var.machine_type, "e2-")
    error_message = "Only E2 machine types are allowed in this lab,
for example e2-micro or e2-medium."
  }
}
```

Test it with a value from another family:

```
terraform plan -var="machine_type=n2-standard-2"
```

The plan fails with your custom error message. Run terraform plan without the override and it succeeds again, showing no changes.

Mark a value as sensitive (optional)

Adding sensitive = true to a variable or output hides its value in plan and apply output. Try it on the external_ip output, apply, and notice the value now shows as sensitive. terraform output -raw external_ip still prints it when you ask for it directly. Remove the line afterwards.

Task 9. Destroy the infrastructure

Destroy uses the same variable values as apply, so it finds the right project and zone without extra input:

```
terraform destroy
```

Review the plan (every resource marked with -), type yes, and check that the VM list in the console is empty.

File layout used in this lab

File	Contents
main.tf	Provider settings and the VM resource, written only in terms of variables
variables.tf	Declarations: name, type, description, default and validation for each input
terraform.tfvars	Actual values for this environment, loaded automatically
prod.tfvars	Alternative values, loaded only with -var-file
outputs.tf	Values Terraform reports after apply

Variable types used

Type	Example variable	Example value
string	machine_type	"e2-medium"
list(string)	network_tags	["web", "dev"]
map(string)	labels	{ env = "dev", owner = "student" }

Review questions

1. Why does project_id have no default value?
2. If machine_type is set in terraform.tfvars and also passed with -var, which value does Terraform use?
3. Why did changing instance_name produce -/+ in the plan while changing tags produced ~?
4. What is the advantage of a validation block over letting the Google API reject a bad value?
5. How would you run the same configuration against a second project without editing any .tf

file?

Key points

- Variables turn a configuration into a reusable template: the code describes the shape of the infrastructure, and tfvars files describe a particular copy of it.
- Keep declarations in variables.tf and values in tfvars files, so reviewers can see what is configurable separately from what is configured.
- Command-line options outrank files, and files outrank environment variables and defaults.
- Validation catches mistakes during planning, before any API call.
- Outputs give you the values you need after an apply without searching the console.